Modelo Relacional

Introducción

Como vimos previamente el modelo de entidad/relación sirve para modelar los datos en una categoría conceptual (primera abstracción), sin hacer referencia a ningún DBMS en particular.

Ya discutimos los pasos que sigue un buen diseñador, y justamente el tercero corresponde a realizar la implementación, pasando del diseño conceptual al diseño lógico (mapeo de modelo conceptual al de implementación/representación).

A continuación analizaremos un modelo de implementación, **el modelo relacional**.

1. Características del Modelo Relacional

Fue propuesto por Codd (*A relational model of data for large shared data banks*, CACM, 1970), investigador de IBM. Este modelo está sustentado por teoría matemática.

Caracteriza al mini-universo como una colección de relaciones (que no tiene el mismo significado que la palabra relación del modelo entidad/relación antes visto). Muchos DBMS comerciales y *open source* se basan en este modelo (entre otros: DB2 y Oracle).

También se maneja la idea de atributo, igual que en el modelo de entidad/relación.

El dominio de un atributo se usa para indicar los valores posibles que puede tomar cierto atributo. Aunque originalmente los valores de un dominio (Codd 69/70) eran valores conocidos para cierto atributo, luego se extiende la idea para permitir almacenar valores "no conocidos". Tomando dicha idea generalizada de dominio se considera que siempre existe un valor en el dominio de un atributo llamado *null* *(este valor se garantiza que no es ninguno de los valores existentes. Ej: si se elige como dominio de la edad valores enteros entre 0 y 150, entonces se agrega a este dominio el valor null)* .

El valor *null* puede usarse para indicar tres conceptos diferentes:

- que el valor de cierto atributo "no es aplicable". Ejemplo: el valor del atributo "apartamento" en el caso de una dirección que sea una casa.
- que el valor de cierto atributo "es aplicable pero es desconocido". Ejemplo: el valor del atributo teléfono de una persona que en el momento de completar los datos se sabe que posee uno pero no se recuerda su número.
- que el valor de cierto atributo "ni siquiera se sabe si es aplicable". Ejemplo: el valor del atributo teléfono de una persona que en el momento de completar los datos se ignora si tiene alguno.

Como se observa, debería haber distintos tipos de *null* para marcar la diferencia, pero el modelo relacional y sus implementaciones concretas no cuenta con esta posibilidad.

Dominio

Es un conjunto de valores atómicos (no se pueden dividir). Incluye el valor especial *null*.

Silvia Alicia Gómez & Leticia Irene Gómez

Aclaración

Ya podemos destacar dos diferencias respecto del modelo ER:

- En el modelo ER se representan entidades y relaciones. En el modelo relacional sólo se usan relaciones (y no tienen el mismo significado que en el otro modelo).
- En el modelo ER los atributos pueden ser compuestos. En el modelo relacional NO.

1.1. Esquema de Relación

Esquema de Relación R(A1, A2,... An)

Está dado por su nombre **R** y la lista de atributos **A1, A2,... An**, donde cada **Ai** es el nombre del rol que juega algún dominio **D** en dicho esquema.

Nótese que al decir "algún dominio D" se está indicando precisamente que varios atributos pueden tener el mismo dominio pero cada uno de ellos está mostrando un rol distinto en el esquema. Por ejemplo, pueden aparecer dos atributos como *sueldo* y *porcentaje*, ambos con dominio **R** $\cup$ **{null}**, pero el primero mostrando el rol de los haberes que se perciben y el segundo el porcentaje de descuentos mensuales.

1.2. Relación para un Esquema de Relación

Relación r para un Esquema de Relación R (también llamado Instancia r del Esquema de Relación R)

Una relación r es un subconjunto del producto cartesiano de los **N** dominios que definen **R**. Si dichos dominios se llaman **D1, D2,... Dn**, entonces $\mathbf{r \subseteq D1 \times D2 \times ... \times Dn}$

El producto cartesiano especifica todas las combinaciones posibles entre los valores de los dominios. Si la cardinalidad de un dominio Di es | Di |, entonces la máxima cantidad de elementos que puede tener una relación sería |D1| * |D2| *... * |Dn|

Un esquema de relación suele ser estático y sirve para caracterizar a las relaciones. Las relaciones de dicho esquema son las que permiten ir reflejando los cambios del mundo real. Cada una de las relaciones es como una foto instantánea del mini-universo.

Podríamos volver a aplicar el concepto de intensión y extensión.

Silvia Alicia Gómez & Leticia Irene Gómez

Muy Importante

Una *relación* puede ser *representada* por una tabla bidimensional de valores:

- Cada fila o tupla representa la colección de valores relacionados.
- Los valores de una columna pertenecen al mismo dominio.

El *esquema de relación* estaría *representado* por el encabezamiento de las tablas:

- El nombre de la tabla (nombre de la relación) y de cada columna (nombre del atributo) ayuda a interpretar el significado de los valores de cada fila.

Notar que tanto las **entidades como las relaciones del modelo E/R** se representan de igual manera a través de una **relación en el modelo relacional**:

- un conjunto de entidades o conjunto de relaciones del modelo de E/R (intensión) es representado como un esquema de relaciones en el modelo relacional (headers de las tablas).
- las entidades o relaciones del modelo E/R (extensión) se representan como conjunto de tuplas en el modelo relacional (tabla).
- una entidad o relación en particular del modelo E/R se representa por una fila o tupla en el modelo relacional.

Podemos interpretar un esquema de relación como una ***afirmación (assertion)***. Así es como cada tupla que pertenece a una relación se entiende como valores que "satisfacen a dicho predicado" (sean entidades o relaciones del modelo E/R). Análogamente a lo que ocurre en inteligencia artificial una relación dice lo que está pero no lo que no se encuentra presente.

Como una relación es un conjunto (ver definición) entonces ***NO puede tener elementos repetidos***.

1.3. Diferencia entre Relación y Tabla

Podemos preguntarnos si existe alguna diferencia formal entre una relación definida como el subconjunto del producto cartesiano y una tabla: ¿Hay orden en las tuplas que forman la relación? Si pensamos en la relación como un conjunto, entonces no hay orden. Si la imaginamos como una tabla, sí lo hay pero sólo por una cuestión gráfica. En realidad no hay preferencia para ordenar las filas de alguna forma. Consideraremos el término relación equivalente a tabla, asumiendo que una tabla es idéntica a otra siempre si y sólo si difieren en el orden de sus filas.

1.4. Restricciones de Modelo Relacional

Como vimos previamente es mejor cuanto antes representar la semántica de lo datos de una aplicación. El modelo de E/R permitía representar ciertas restricciones y otras no. Ahora veremos cuáles son las restricciones que pueden expresarse en el modelado lógico relacional.

Silvia Alicia Gómez & Leticia Irene Gómez

De ahora en más denotaremos una convención de notación (sensible a mayúsculas/minúsculas) para utilizar en las definiciones y teoremas.

- Los esquemas de relación se notan R, S, T
- Los atributos de un esquema de relación se denota con A, B, C, A1, A2,... An
- Las instancias de los esquemas o sea las relaciones se notan r, s
- Los valores de los atributos a, b, c, a1, a2,... an
- Las filas o tuplas se denotan con t, u, v, w (o agregándoles subíndices)
- Los conjuntos de atributos X, Y, Z. También podemos usar concatenación de atributos simples como por ejemplo ABC en vez de {A, B, C}, etc.
- La restricción de una fila o tupla a un conjunto de atributos es t[X] (en vez de t se puede usar cualquiera de las letras para tuplas, y en vez de usar X se puede utilizar cualquiera de las letras designadas para conjuntos de atributos).

1.4.1. Restricciones de Dominio

Todo atributo debe ser atómico en su dominio.

1.4.2. Restricciones de Clave

En la definición conjuntista no hay elementos repetidos, o sea todos los elementos deben ser distintos. Como una relación es un conjunto, entonces las tuplas deben distinguirse entre sí. Esta propiedad debe ser **invariante en el tiempo**, o sea por más que se agreguen, eliminen o cambien los valores de la tuplas debe seguir verificándose. No hay dos tuplas cuyas combinaciones de valores de atributos coincidan.

Por esta restricción podemos asegurar que hay un conjunto de atributos en el esquema de relación que verifica la propiedad de que en cualquiera de sus instancias no existen dos tuplas que repitan sus valores.

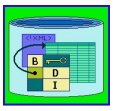
Superclave (superkey)

Un subconjunto **X** de atributos es superclave si y sólo si toda instancia **r** verifica que si **t1** y **t2** son tuplas de dicha instancia y **t1** $\neq$ **t2**, entonces **t1[X]** $\neq$ **t2[X]**.

Ahora bien, todo subconjunto de atributos del esquema de relación con esta propiedad se denomina ***superclave***. Es seguro que hay una superclave formado por todos sus atributos, pero interesa una identificación más económica: el concepto de clave candidata permite diferenciar las tuplas utilizando menos información.

Una **clave candidata Y** de un esquema de relación es una superclave de dicho esquema que verifica la propiedad de que si se remueve cualquiera de sus atributos se obtiene un subconjunto de atributos que no es superclave de dicho esquema, o sea, es **minimal** respecto de la propiedad de ser superclave.

Silvia Alicia Gómez & Leticia Irene Gómez



Por ejemplo, en el esquema de relación *alumno* el conjunto de atributos *{legajo, nombre}* es superclave pero no clave candidata. En cambio el atributo *legajo* es superclave y clave candidata.

Un esquema de relación pueden tener muchas claves candidatas

En la tabla se marca la clave subrayando los atributos que la forman. Si hubiera más de una clave candidata no sirve la notación del subrayado. En ese caso se las lista por separado y a lo sumo se puede subrayar una clave. Pero en general, se prefiere utilizar el subrayado cuando la clave candidata es única.

Ejemplo: Una clave candidata se define sobre un esquema de relación, ya que toda instancia (relación) en el tiempo debe satisfacer la restricción de clave. Pero supongamos que tenemos la siguiente instancia y queremos encontrar cuáles son sus superclaves, cuáles son su claves candidatas y a partir de ahora (y para cualquier operación que se realice sobre la misma en el futuro) garantizar la restricción de clave. Analizar cuáles serían las claves candidatas a definir.

Atr1	Atr2	Atr3
1	A	X
2	A	Y
3	B	Z

Un análisis detallado nos lleva a las siguientes conclusiones:

- Atr1 + Atr2 + Atr3 es superclave (obvio, sino no sería una relación en el modelo relacional)
- Atr1 + Atr2 es superclave
- Atr1 + Atr3 es superclave
- Atr2 + Atr3 es superclave
- Atr1 es superclave
- Atr2 NO es superclave ya que **t1[Atr2] = t2[Atr2] y t1 ≠ t2**
- Atr3 es superclave

Cuáles de las superclaves anteriores pueden ser claves candidatas?

- Atr1 + Atr2 + Atr3 NO puede ser clave candidata porque contiene Atr1 que es superclave (no es minimal), o sea al remover Atr2+Atr3 obtengo una superclave.
- Atr1 + Atr2 NO puede ser clave candidata porque contiene Atr1 que es superclave (no es minimal), o sea al remover Atr2 obtengo una superclave.
- Atr1 + Atr3 NO puede ser clave candidata porque contiene Atr1 que es superclave (no es minimal), o sea al remover Atr3 obtengo una superclave.
- Atr2 + Atr3 NO puede ser clave candidata porque contiene Atr3 que es superclave (no es minimal), o sea al remover Atr2 obtengo una superclave.
- Atr1 es clave candidata
- Atr3 es clave candidata

Aunque normalmente se identifican y definen las claves candidatas ANTES de poblar una relación con tuplas (se identifica semánticamente la propiedad de ser clave), en este caso partiendo de una instancia concreta hemos identificado qué claves candidatas puede definirse a partir de ahora.

Silvia Alicia Gómez & Leticia Irene Gómez

1.4.3. Esquema e Instancia de Base de Datos Relacional

Hasta ahora hemos discutido qué son las relaciones y cuáles son sus características, pero no podemos perder de vista que la representación de un mini-universo no se hace especificando uno o dos esquemas de relación, sino con un ***conjunto de esquemas de relaciones*** más ***un conjunto de restricciones*** que aseguren la integridad que tendrá la información que luego se manipule. Esto es justamente lo que debe realizar el diseñador en esta etapa y es nada más y nada menos que la especificación de la intensión para la base de datos específica para determinado sistema. Más formalmente vamos a definir:

Esquema de Base de Datos Relacional

Conjunto de esquemas de relación **{R1, R2,...Rn}** y un conjunto de restricciones de integridad **I**.

Por supuesto que cuando comience a usarse el sistema, se deberán contemplar la modificación de la información que se almacene, con el fin de representar los cambios del universo. Pero gracias al conjunto de esquemas de relación diseñado, se crean instancias de cada una de las relaciones por medio de operaciones de inserción, borrado, y actualización de tuplas.

Las restricciones de integridad definidas en la intensión deben seguir valiendo mientras cada uno de los esquemas de relación no cambie.

Instancia de un Esquema de Base de Datos Relacional

Conjunto de relaciones **{ r1, r2,... rn }** donde cada **ri** es instancia del esquema **Ri** con $1 \leq i \leq n$ y donde todas esas relaciones satisfacen las restricciones de integridad **I**.

De las definiciones antes citadas queda claro lo que son **R** y **r**, pero falta definir cuáles son las restricciones de integridad de datos que deben definirse y verificarse:

1.4.4. Restricción de Integridad de Entidad

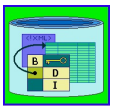
Recordar que el término entidad no tiene el mismo significado que en el modelo E/R.

Esta restricción asegura que ningún valor de clave puede ser *null*. Por lo tanto un esquema de relación, debe garantizar por adelantado que no haya relación que posee alguna tupla donde algún valor de atributo que compone su clave pueda ser *null*.

Se supone que los atributos que forman una clave son precisamente aquellos que individualizan una tupla (representativa de la realidad). Si alguno de los valores fuera ***null***, entonces alguno de los tres próximos razonamientos sería cierto:

- ☒ "Dicho atributo no es aplicable", lo cual sería imposible ya que el atributo que forma la clave es elegido por determinar identidad.
- ☒ "Es aplicable, pero no se sabe cuanto vale", lo cual sería absurdo porque de no conocer su valor es imposible saber qué parte de la realidad realmente representa dicha tupla.
- ☒ "No se sabe si es aplicable", absurdo por la misma razón anterior

Silvia Alicia Gómez & Leticia Irene Gómez



1.4.5. Restricción de Integridad Referencial

Se especifica entre dos esquemas de relaciones y se usa para garantizar consistencia entre ambas. Consiste en definir **claves foráneas** (dependencias de inclusión).

Clave Foránea (Foreign Key)

Un conjunto de atributos **X** de un esquema de relación **R** es una ***clave foránea de R*** si satisface las siguientes dos condiciones:

- Los atributos de **X** tienen el mismo dominio que alguna **clave Y** de algún otro esquema de relación **S**.
- Se verifica que siendo **r** una relación de **R** y **s** una relación de **S**, entonces para toda tupla **t** de **r** existe una tupla **u** de **s** tal que **t[X]=u[Y]** o bien t[X] es null.

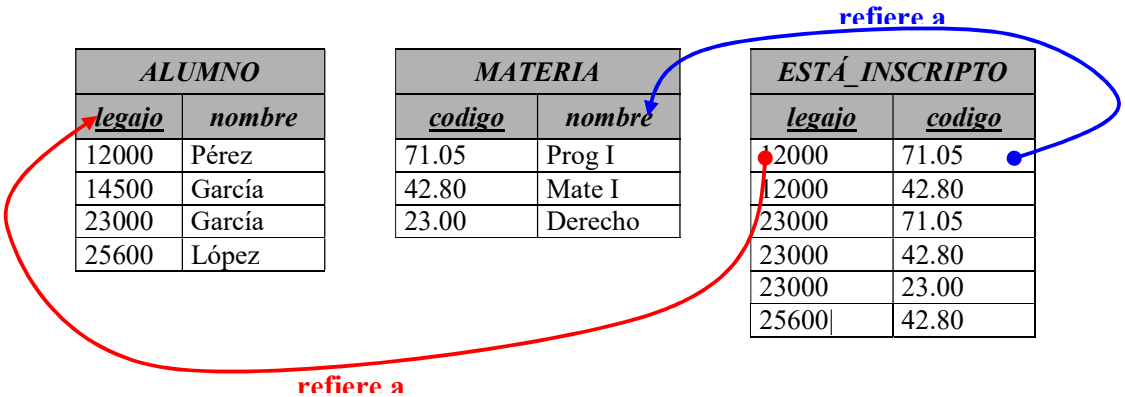
Se dice que el conjunto de atributos X referencian o refieren al esquema de relación S.

Aclaración

Con respecto a la condición de que **t[X]** puede ser null, no todos los autores están de acuerdo porque puede traer anomalías.

Ejemplo:

En el siguiente esquema ***legajo*** es clave foránea de ***ESTA_INSCRIPTO*** y refiere al esquema de relación ***ALUMNO.*** Además ***codigo*** también es clave foránea de ***está_inscripto,*** pero refiere al esquema de relación ***materia.***



ALUMNO	
legajo	***nombre***
12000	Pérez
14500	García
23000	García
25600	López

MATERIA	
codigo	***nombre***
71.05	Prog I
42.80	Mate I
23.00	Derecho

ESTÁ_INSCRIPTO	
legajo	***codigo***
12000	71.05
12000	42.80
23000	71.05
23000	42.80
23000	23.00
25600	42.80

Existen otras restricciones muy importante sobre las dependencias de los datos, que veremos más adelante: ***Restricciones de Dependencias: Funcionales, Multivaluadas, de Junta.***

Silvia Alicia Gómez & Leticia Irene Gómez

1.5. Operaciones

Dado que en esta etapa usamos un DBMS particular para implementar determinada base de datos, es importante analizar qué operaciones deberían tener los lenguajes DDL y DML en el modelo relacional.

1.5.1. Lenguaje DDL

A partir del diseño de un esquema de base de datos relacional hay que crear las tablas involucradas y definir las restricciones de integridad de datos.

1.5.2. Lenguaje DML

Ahora debemos contar con operaciones para manipular los datos: realizar inserciones, eliminación, y actualización de tuplas, y por supuesto permitir realizar consultas para recuperar la información almacenada. De las consultas nos vamos a ocupar más adelante.

Vamos a analizar genéricamente (no un lenguaje relacional específico) qué debería permitirse y qué problemas podrían ocurrir en la actualización de los datos de la base de datos. Tener en cuenta que las restricciones siempre deben cumplirse por tanto el DBMS debe rechazar alguna operación que quiera violarla o tomar alguna acción (automática o con interacción de usuario) que coloque a la instancia en un estado que cumpla todas las restricciones.

◆ *Operación de Inserción*

Se usa para insertar nueva tupla **t** en una relación **r**, instancia de **R**. Para ello se provee una lista de los valores de atributos para la nueva tupla.

La inserción podría violar las 4 restricciones citadas previamente:

- **Restricción de dominio**

  Se viola cuando la tupla a insertar tiene algún valor de atributo que no aparece en el dominio de dicho atributo.

- **Restricción de clave**

  La restricción se viola cuando ya existía una tupla **u** de **r** tal que **t[X]=u[X]** y **X** era clave de **r**, es decir, intento insertar una clave repetida.

- **Restricción de integridad de entidad**

  Se viola si algún valor en los atributos de la clave para la tuplas **t** es null.

- **Restricción de integridad referencial**

  Se viola si el valor de alguna clave foránea en **t** refiere a alguna tupla que no existe en la relación referenciada

Silvia Alicia Gómez & Leticia Irene Gómez

Ahora bien, si la inserción viola una o más restricciones, ¿qué se puede hacer?

- Rechazar la inserción.
- Informar de error e intentar repararlo (preguntando o sugiriendo soluciones). Notar que esta solución para la violación de restricción de integridad referencial es complicada porque implica sugerir todas las posibles tuplas que podría estar referenciando t, etc.

◆ *Operación de Borrado*

Se usa para eliminar una/varias tupla/s en una relación **r**, instancia de **R**. Para ello se proveen condiciones sobre los atributos para que la/s misma/s pueda/n ser identificadas/s en **r**.

La eliminación sólo podría violar:

➢ **Restricción de integridad referencial**

Se viola si la/s tupla/s a borrar está/n referenciada/s por otras tuplas de alguna relación en la base de datos.

Las acciones que se pueden seguir si el borrado viola dicha restricción son:

- Rechazar el borrado. Este tipo de acción de denomina **ON DELETE NO ACTION**.
- Intentar un borrado en cascada, o sea borrar la/s tuplas que referencian a la/s que se quiere/n eliminar, y seguir este proceso recursivamente (hay que navegar por otras relaciones). Este tipo de acción de denomina **ON DELETE CASCADE**.
- Modificar los valores de las claves foráneas que referencia/n a la que se quiere eliminar haciendo que referencien a otra tupla (o a null si se acepta este valor), quizás con alguna interacción del usuario. Este tipo de acción de denomina **ON DELETE SET**.

◆ *Operación de Modificación*

Se usa para cambiar los valores de ciertos atributos en una/varias tupla/s en una relación r, instancia de R. Para ello se proveen condiciones sobre los atributos para que la/s misma/s pueda/n ser identificadas/s en r y por supuesto también se especifican cuáles son los nuevos valores de atributos.

Queda para el lector, en base a los casos vistos en la inserción y el borrado, pensar cuáles son los casos de violación para la modificación y cuáles son las acciones a tomar.

Silvia Alicia Gómez & Leticia Irene Gómez